

Atuando na Indústria: Fundamentos de Engenharia de Software e DevOps para Bioinformatas

Breno de Andrade Tauil

Sobre mim

Biólogo especialista em biotecnologia pela UERJ

Mestrando em Bioinformática na USP

Engenheiro de Bioinformática na Gen-t

+3 anos trabalhando como análista de dados e
preço para comercializadoras de energia



Cronograma

Dia 1

Git

Básicos de Eng. Software

Desenvolvimento Orientado a Testes

Dia 2

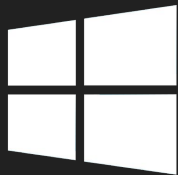
DevOps

Containers

Orquestradores

Mini-projeto

Instalação



<https://git-for-windows.github.io/> e siga as instruções padrões



```
git --version
```

O sistema solicitará o Xcode

```
brew install git
```



```
sudo apt update && sudo apt install git -y
```

Discussão

"Já teve um script que funcionava mas que ninguém mais conseguia usar?"

"Qual o custo disso?"

"O que faz um script “órfão”?"

GIT



Controle de Versão



É como se fosse salvar em um video-game.

Você quer salvar seu progresso para caso você “morra”, você não perde seu progresso.

Com git frequentemente você vai usar um comando para salvar o seu progresso no seu computador.



Controle de Versão



Um projeto com vários arquivos e diferentes linguagens/extensões.

Se eu deletei um desses arquivos, eu perco meu progresso.

Com o git você pode recuperar seus arquivos, como se fosse um *checkpoint*.

```
Test/  
├── Main Directory/  
│   ├── .Dir_Store/  
│   │   └── TestFile.txt  
│   ├── Directory 1/  
│   │   └── Directory 2/  
│   │       ├── Directory 3/  
│   │       │   └── Directory 4/  
│   │       │       └── Hello World.txt  
│   │       └── Say World.txt  
│   ├── Directory A/  
│   │   ├── .hiddenFile.txt  
│   │   └── Hmm.txt  
│   ├── directory-tree-print.cpp  
│   ├── letseee.txt  
│   └── printTree.exe  
├── v0_0_2_Test.py  
└── v0_0_3_Test.py
```


Comandos



```
git init
```

Inicia o repositório

```
git add [arquivos]
```

Seleciona os arquivos para salvar

```
git commit -m "[sua mensagem]"
```

De fato salva os arquivos selecionados

```
git log
```

Abre o log com suas mudanças

```
git checkout [branch]
```

Git $\neq$ GitHub



Git é o seu cartão de memória.



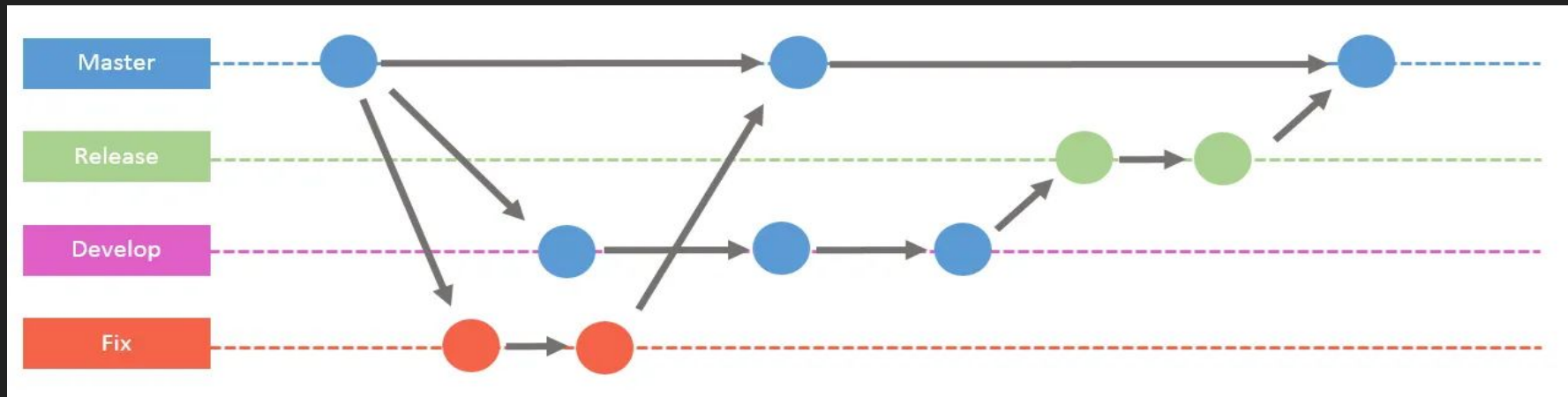
Github é um site onde você pode compartilhar seus arquivos.



Branches

Permitem que você possa isolar o seu trabalho.

Você vai poder focar em desenvolver novos códigos / features para seu trabalho sem interferir na parte que já está funcionando.





Comandos

```
git branch
```

Mostra todas as branches

```
git branch [nome]
```

Cria uma branch [nome]

```
git checkout [nome]
```

Entra na branch [nome]

```
git checkout -b [nome]
```

Cria uma branch [nome] e entra nela direto

```
git push origin [nome]
```

“Empurra” a sua branch local para o repositório remoto

Comandos



```
git pull origin master
```

“Puxa” as mudanças na nuvem com o seu local

```
git status
```

Mostra o status da sua branch local comparado a outras

```
git merge [nome]
```

Faz o merge da branch [nome] com a atual

Na prática



Crie um fork e uma copia local do repositório do curso

```
https://github.com/btauil/aulaRSG.git
```

Você provavelmente vai ter que configurar uma identidade git local

```
git config user.name "Nome"
```

```
git config user.email "seu@email.com"
```

Na prática



Crie uma branch “comentarios”

```
git _____ _ comentarios
```

Comente o arquivo conteudo_gc.py e faça o commit

```
git ___ _  
git commit __ _____
```

Faça o push das suas alterações para a branch remota

```
git push origin comentarios
```



Boas práticas

Commits frequentes, pequenos (também chamados de commits atômicos) e claros (commits funcionais)

```
git add .  
git commit -m "fix: correção de bug na função X"
```

Use branches com nomes claros

```
git checkout -n issue#1_adicionar_conteudo_gc
```

Sempre faça pull antes de push

```
git pull origin main  
git push origin main
```




Boas práticas

Mantenha repositórios pequenos, use o .gitignore

```
# .gitignore  
superarquivogigantescodedezterias.csv  
.  
.venv  
*.log  
topsecret.txt
```

Revise antes de publicar

```
git status
```



Como usar as Branches corretamente?

Estratégias de branching:

GitFlow

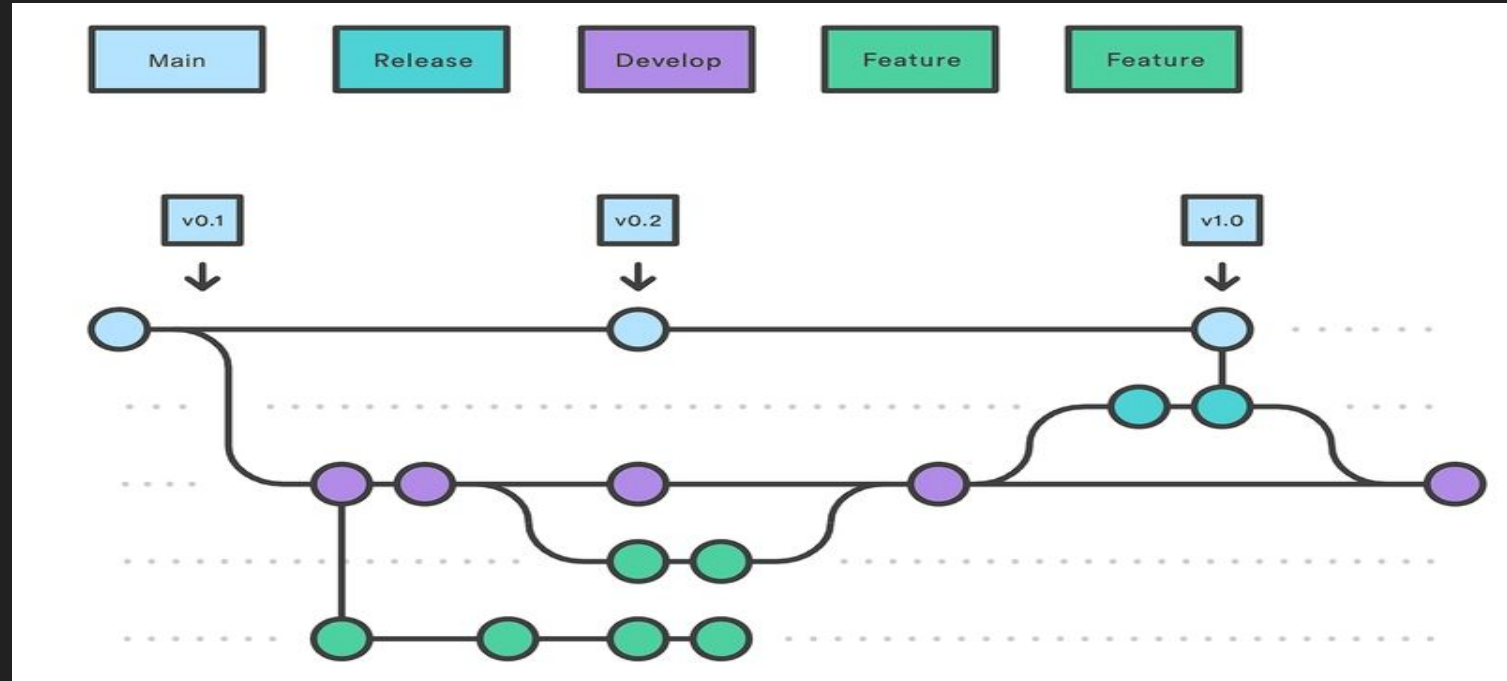
TBD

Release Branching

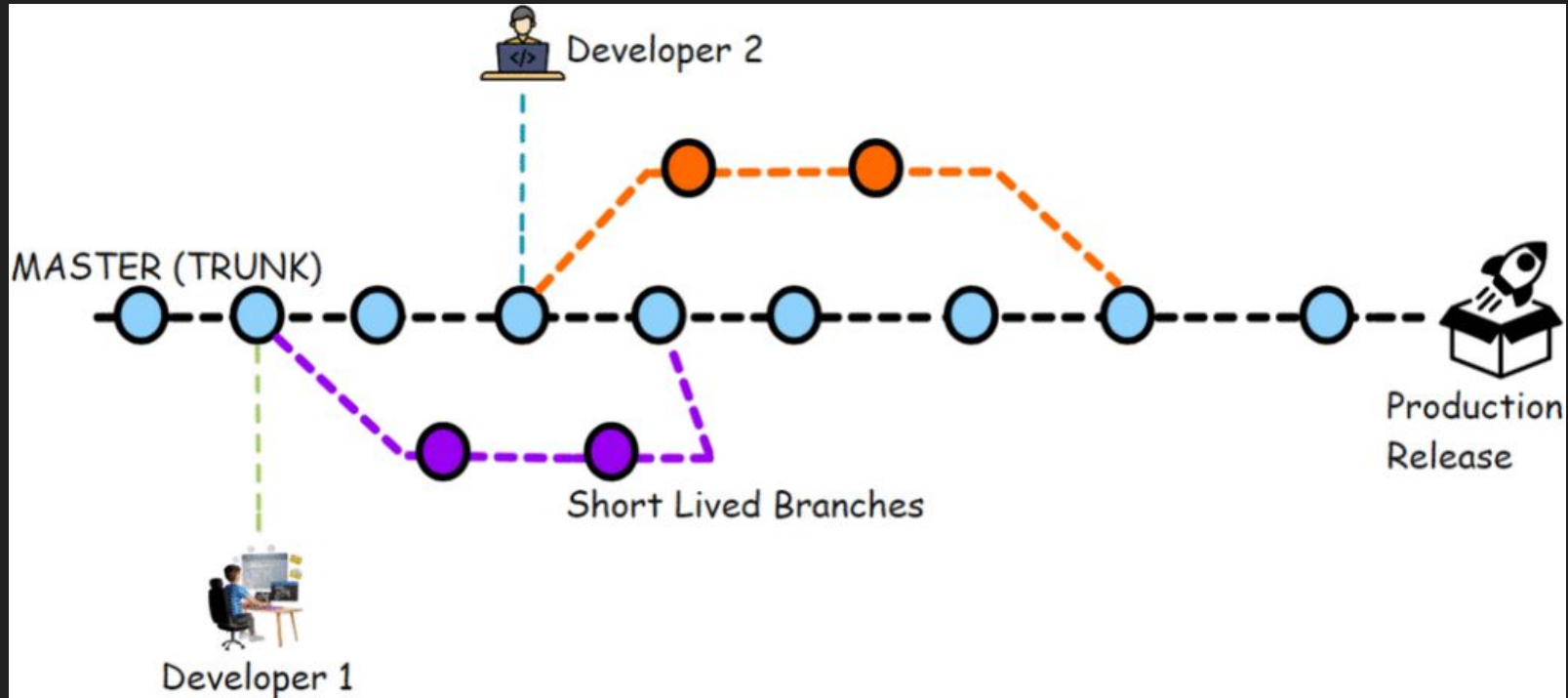
Environment Branchin

etc...

GitFlow



Trunk-Based Development



Engenharia de Software

Engenharia de Software

"É o processo sistemático de desenhar, desenvolver, testar e manter software eficiente, confiável e de alta qualidade"

Aplicar conceitos de engenharia para o desenvolvimento de software

Princípios



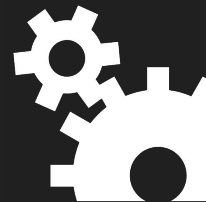
Modularidade



Abstração



Encapsulação



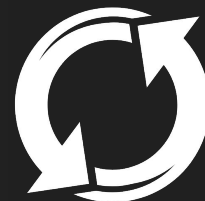
Manutenção



Testes



Padrões de
Design



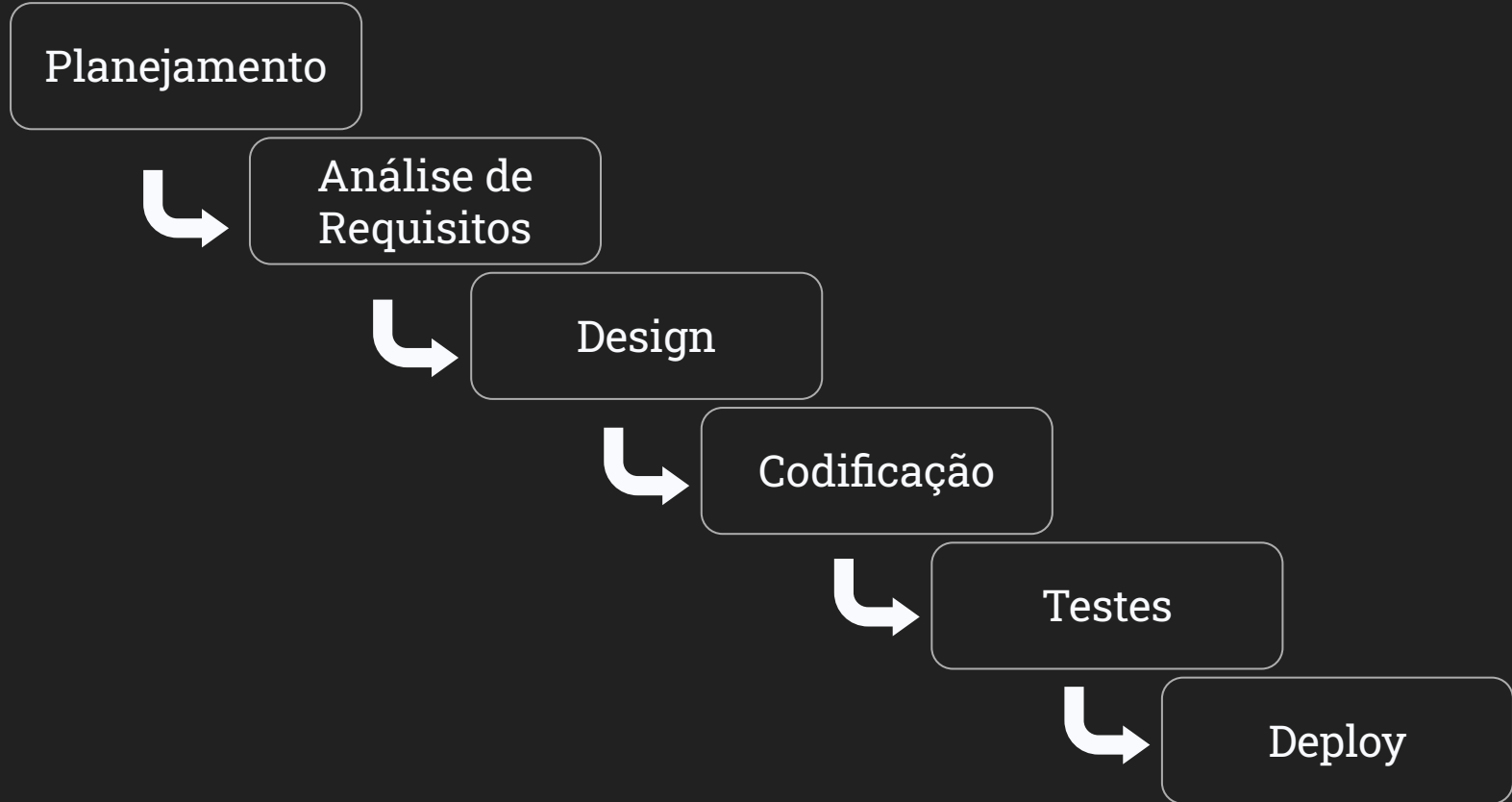
Ágil



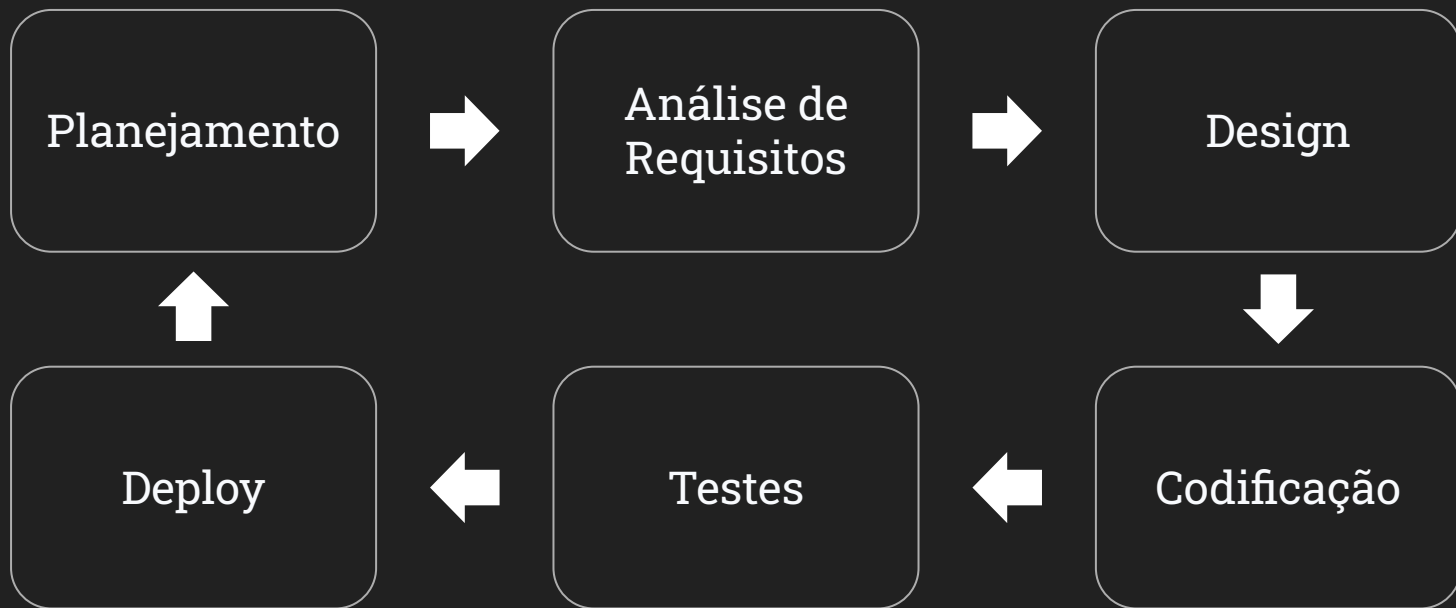
CI/CD

Ciclo de Vida do Desenvolvimento de Software

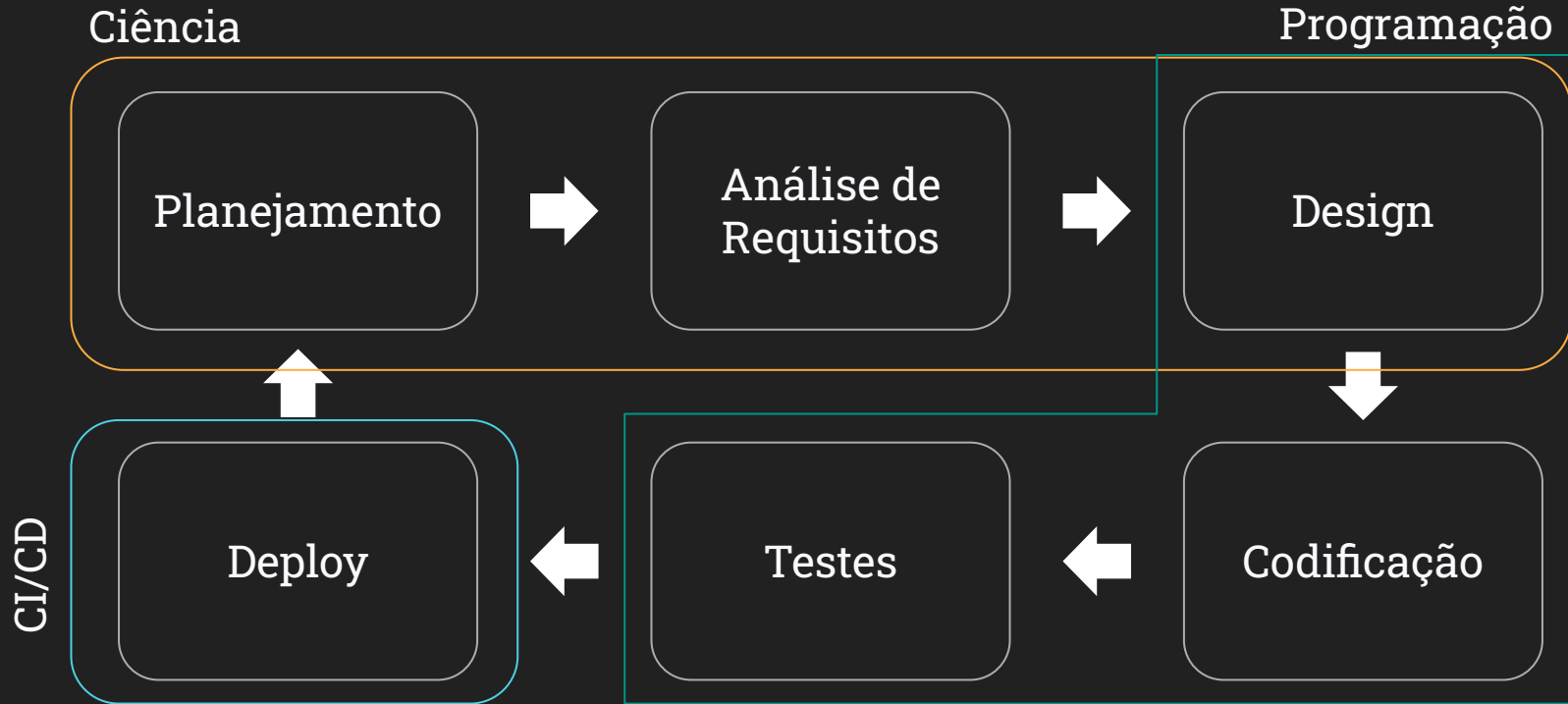
Modelo Cascata



Métodos Ágeis



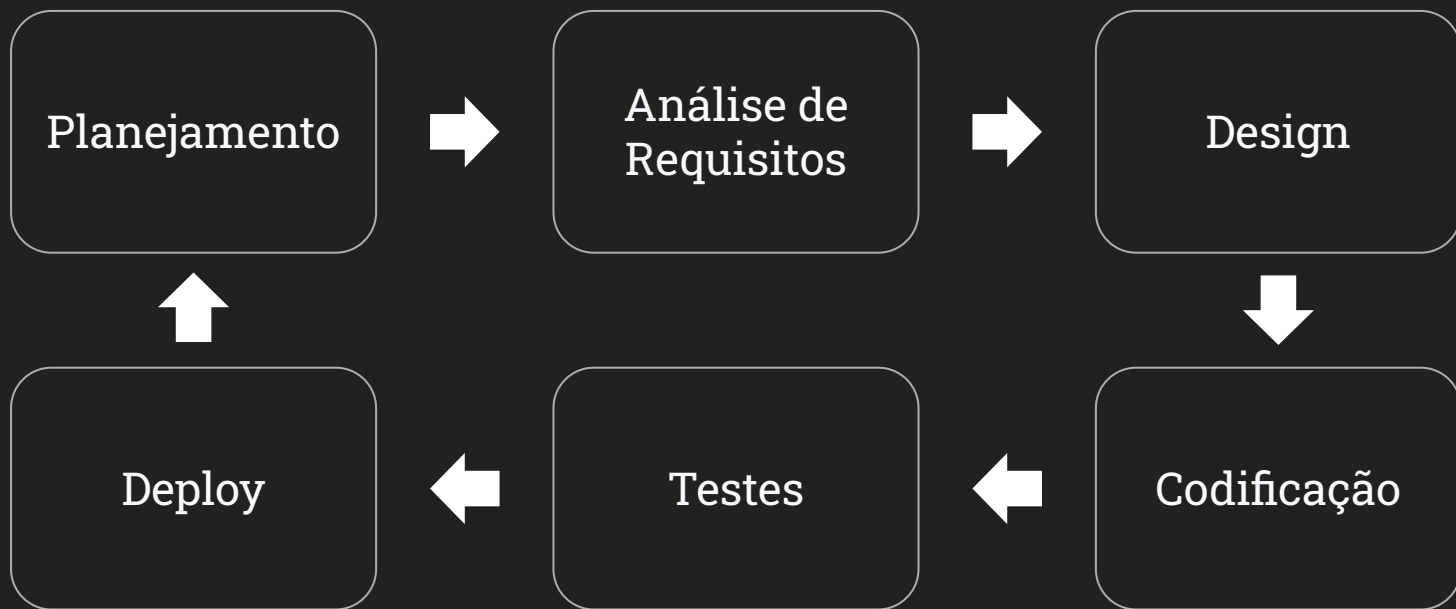
Métodos Ágeis



Produto Viável Mínimo

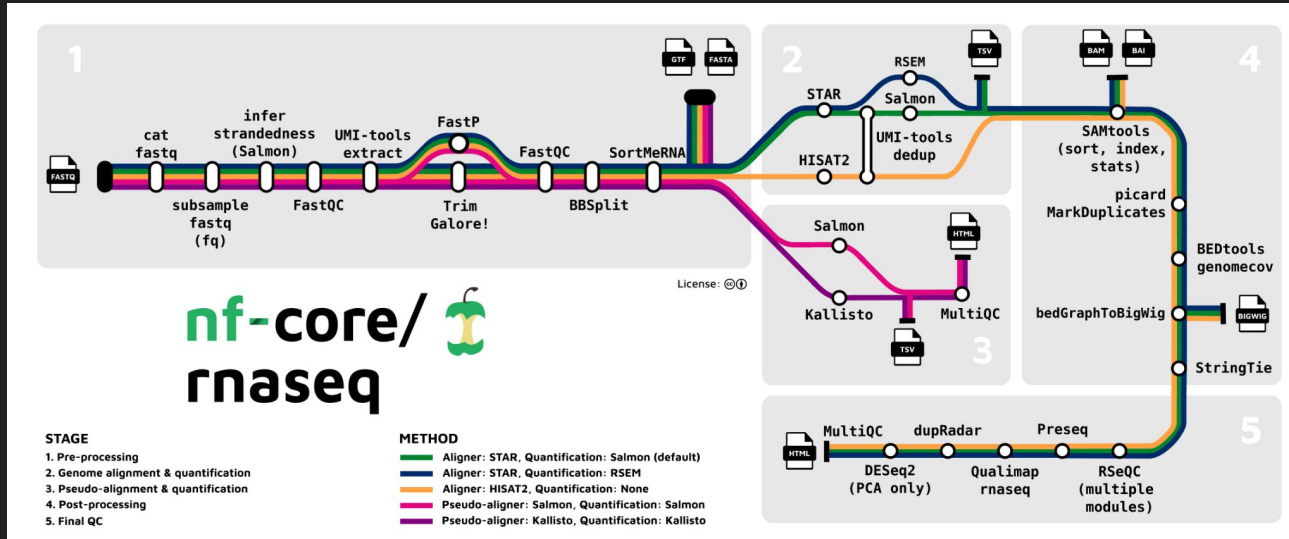
A versão mais simples de um produto que pode ser lançada com uma quantidade mínima de esforço e desenvolvimento.

Produto Viável Mínimo



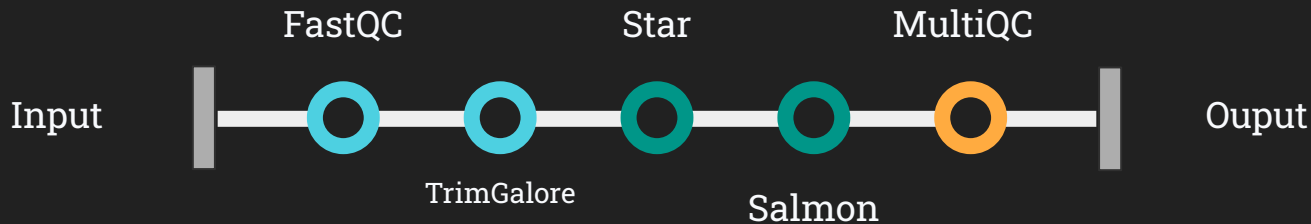
Produto Viável Mínimo

Exemplo de pipeline / produto completo:



Produto Viável Mínimo

Mas talvez a sua pipeline de RNAseq seja mais parecido com isso:



Desenvolvimento Orientado por Testes (TDD)

Escrever apenas o código necessário para passar em cada teste.

Refinar os testes e o código antes de avançar para uma nova funcionalidade.

Red-Green-Refactor

Escreva o teste

Comece escrevendo um teste que só passa se as especificações da sua feature batem -> te força a pensar no sistema antes

Red-Green-Refactor

Escreva o teste

Corra os testes

Testes devem falhar, você não começou a escrever a sua feature ainda -> garante a falha, checa que os testes realmente estão fazendo a sua parte

Red-Green-Refactor

Escreva o teste

Corra os testes

Escreva o código

Escreva o código mais simples para que esses testes passem

Red-Green-Refactor

Escreva o teste

Corra os testes

Escreva o código

Todos os testes devem passar

Garante que as especificações básicas da sua feature rodam corretamente

Red-Green-Refactor

Escreva o teste

Corra os testes

Escreva o código

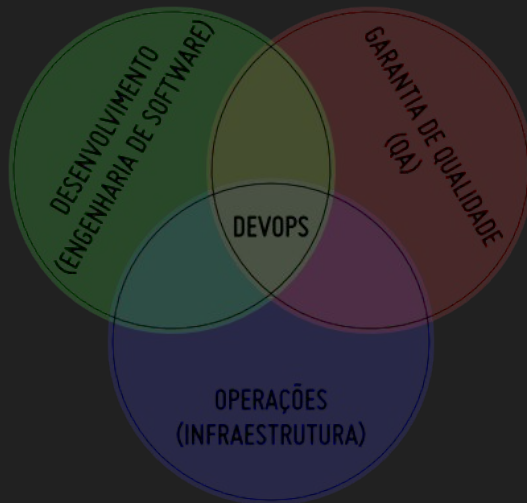
Todos os testes devem passar

Refatorar

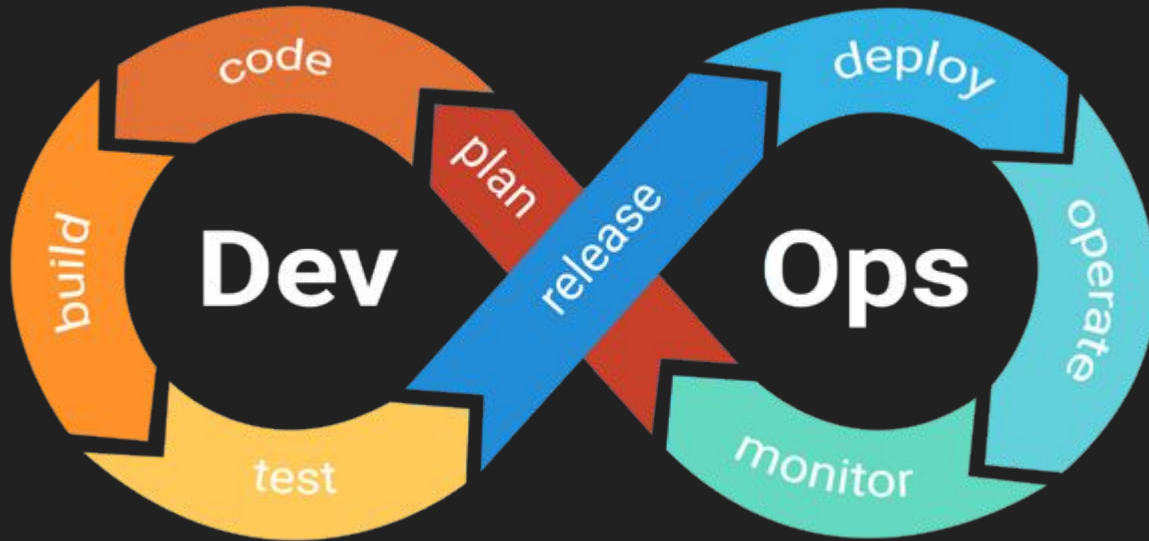
Melhore o código respeitando os testes.

DevOps

É uma cultura na engenharia de software que aproxima os desenvolvedores de software (Dev) e os operadores do software / administradores do sistema (Ops)



DevOps



Miniprojeto

Vamos montar um pipeline de RNAseq que rode dentro de um container

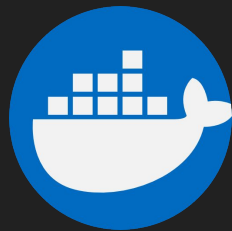


Reprodutibilidade

“Na minha máquina funciona”

- Análises irreprodutíveis
- Versões diferentes
- Dependências não instaladas
- Caminhos absolutos

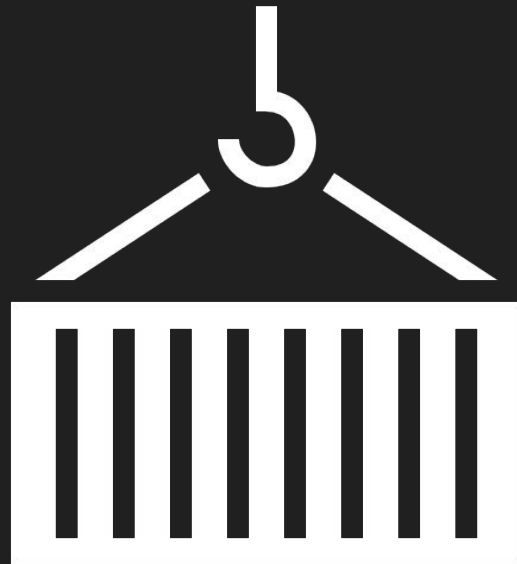
Docker



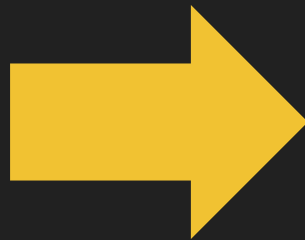
“Container é uma unidade de software empacotada de forma leve e autônoma que contém tudo o que é necessário para executar uma aplicação.”

Cada container é isolada em seu próprio ambiente e compartilha o sistema operacional da máquina hospedeira.

Mais leves do que uma máquina virtual.



Docker



Docker

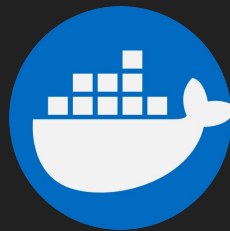


Imagens



Containers

Docker Image



Imagens

Como se fosse uma receita de bolo.
Contém:

- Tecnologias que precisamos
- Runtime
- Ferramentas ou instruções necessárias para rodar nosso código

Docker Container

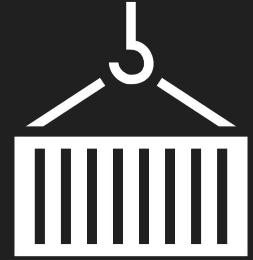


O que vai de fato rodar o código (o que é feito a partir da receita)

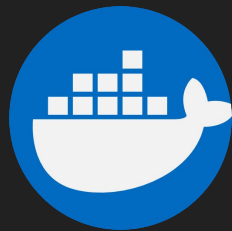


Containers

Docker

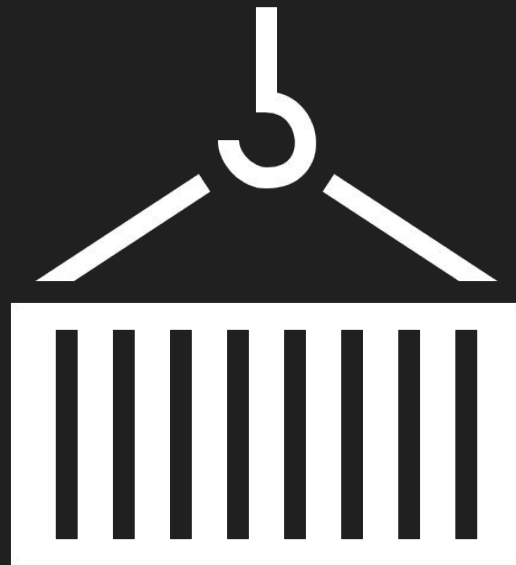


Docker

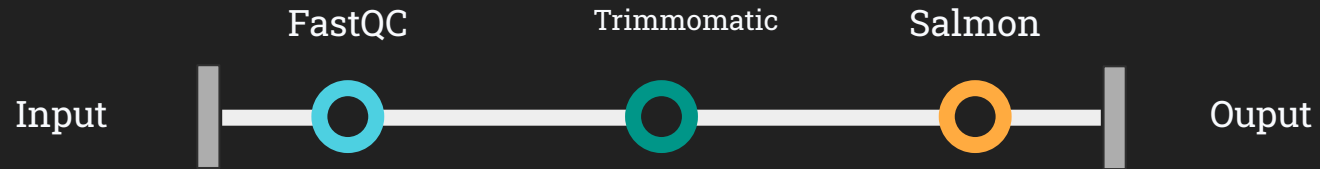


Simplifica o processo de compartilhar código. Um comando é suficiente.

<https://www.docker.com/get-started/>



Planejamento



Análise de Requisitos

FastQC

```
conda install bioconda::fastqc=0.12.1
```

Trimmomatic

```
conda install bioconda::trimmomatic=0.39
```

Salmon

```
conda install bioconda::salmon=1.10.2
```

Dockerfile



micromamba + bioconda + FastQC, Trimmomatic, Salmon

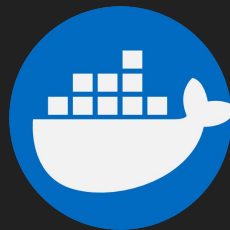
Imagem base com as dependências do micromamba.

```
FROM mambaorg/micromamba:1.5.3
```

Canal bioconda com os binários pré-compilados dos nossos softwares

```
RUN micromamba install -y -n base \  
    -c bioconda -c conda-forge \  
    fastqc=0.12.1 trimmomatic=0.39 \  
    salmon=1.10.2 \  
&& micromamba clean --all --yes
```

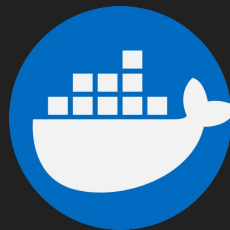
Dockerfile



Instruções Dockerfile

Instruções	Descrição
FROM	Cria um pedaço da imagem a partir de uma imagem base
RUN	Executa um comando

Docker



Agora montaremos a image docker

```
docker build -t rnaseq:0.1.0 . ← Dentro da pasta com a dockerfile
```

-t: Da um nome (tag) pra imagem

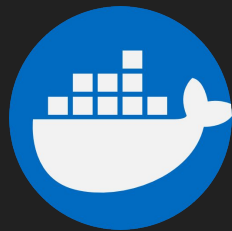
Rodando o container

```
docker run --rm \
  -v $(pwd)/data:/data \
  rnaseq:0.1.0 salmon
```

--rm: Diz ao docker para deletar o container depois

-v: Diz ao docker qual volume usar

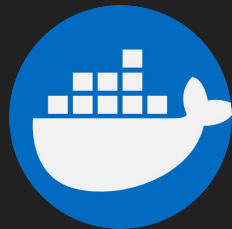
Volume



“Área de armazenamento que atua como o disco do container”

É a pasta com os seus dados onde o container vai poder interagir.

Dockerfile



Ruim — cache invalidado a cada edição

```
FROM ...micromamba  
COPY rnaseq_pipeline.sh .    ← muda sempre  
RUN micromamba install ...  ← re-executa!
```

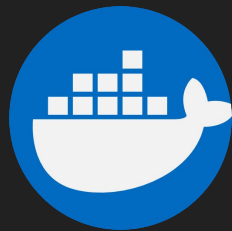
Bom — instalação fica em cache

```
FROM ...micromamba  
RUN micromamba install ...  ← em cache  
COPY rnaseq_pipeline.sh .    ← só isto  
refaz
```

Cada instrução é uma camada.

Mudar uma invalida todas as seguintes.

Instruções



WORKDIR

```
WORKDIR /app
```

Diretório de trabalho do container

COPY

```
COPY rnaseq_pipeline.sh /app/rnaseq_pipeline.sh
```

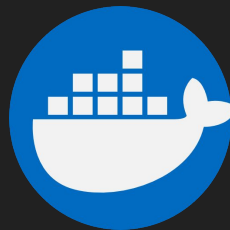
Copia um arquivo para o container

CMD

```
CMD ["bash", "/app/rnaseq_pipeline.sh"]
```

Define comandos padrão

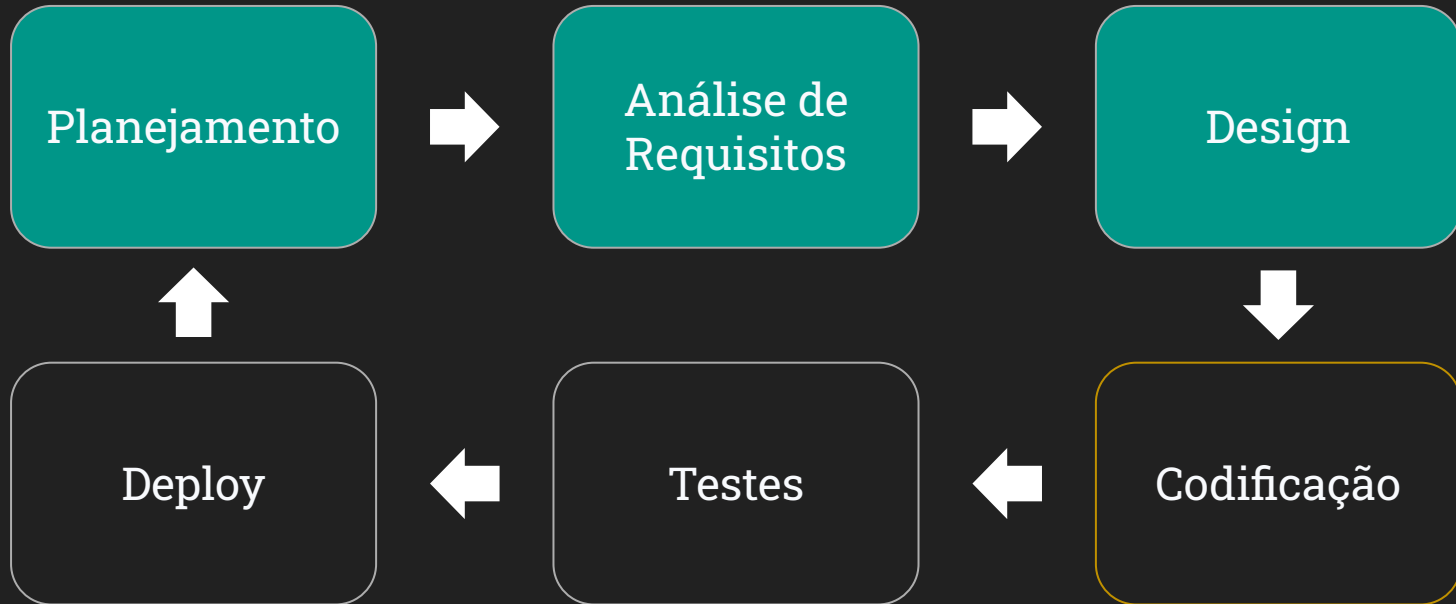
Dockerfile



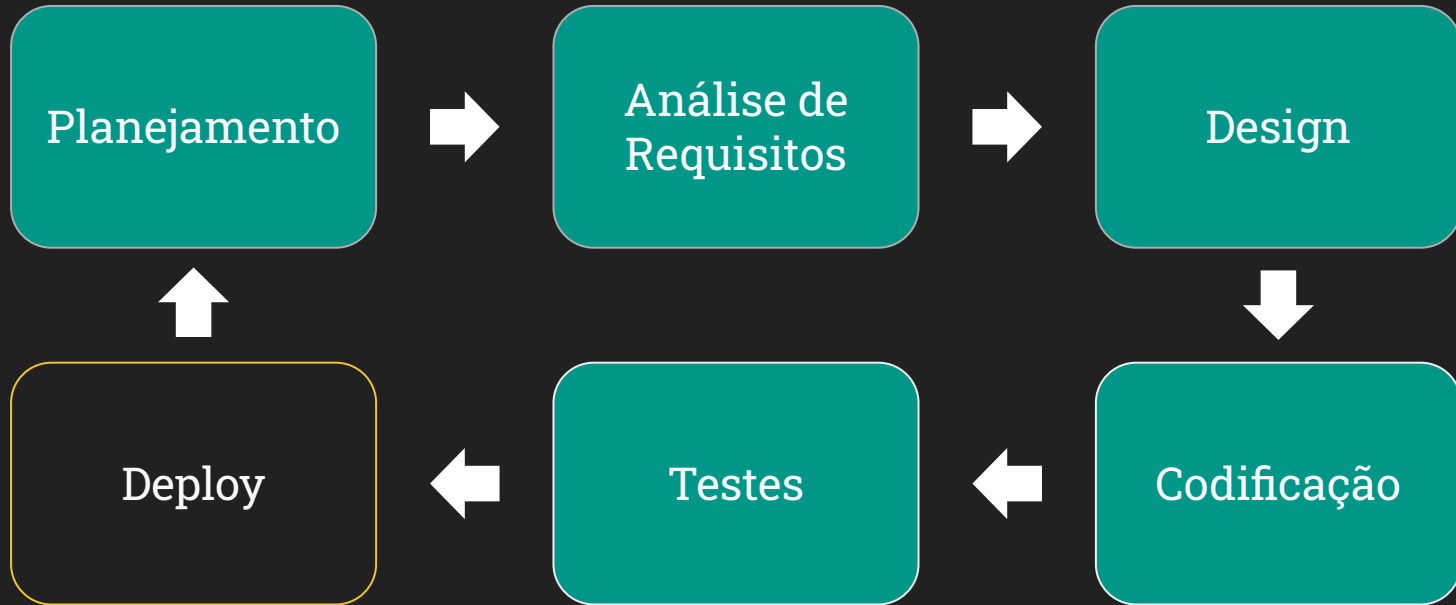
Instruções Dockerfile

Instruções	Descrição
FROM	Cria um pedaço da imagem a partir de uma imagem base
RUN	Executa um comando
WORKDIR	Define o diretório de trabalho
COPY	Copia arquivos e diretórios
CMD	Especifica comandos padrão

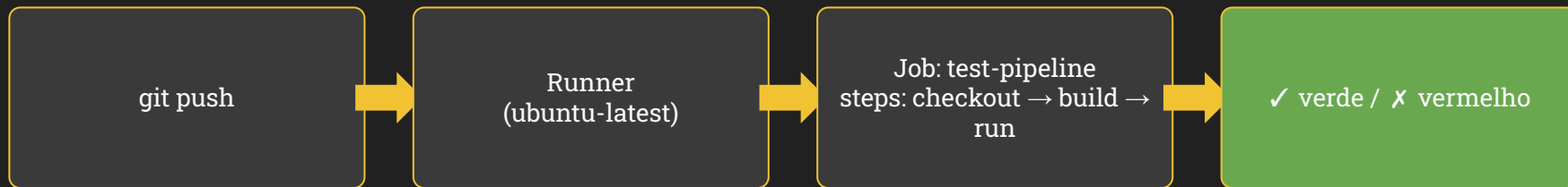
Pipeline RNAseq



Pipeline RNAseq



Integração Contínua (CI)



- **GitHub Actions:** integrado ao GitHub, YAML em `.github/workflows/`.

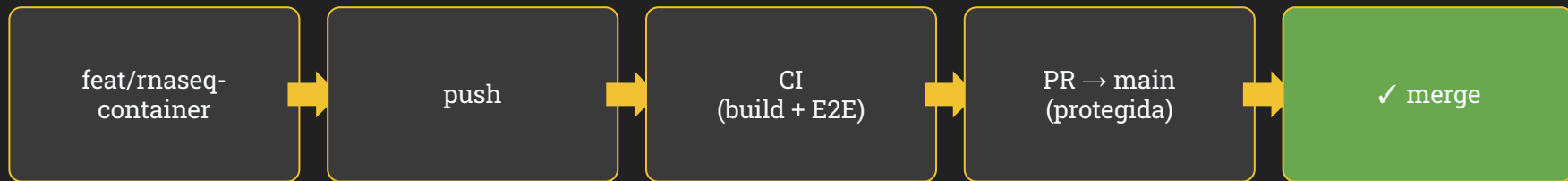
Integração Contínua (CI)

```
jobs:
  test-pipeline:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: |
          cd exercises/project_day2
          docker build -t rnaseq-pipeline .
      - run: |          # cria fastq de amostra
          ...
      - run: |          # roda o pipeline E2E
          docker run --rm -v $PWD/data:/data \
            rnaseq-pipeline /data/sample_1.fastq.gz
```

o Actions não lê workflows de subpastas -> daí o cd.

- **Verde** = o pipeline termina com exit code 0.
- **Teste E2E:** constrói a imagem e roda de verdade num fastq minúsculo.

Integração Contínua (CI)



A main é o nosso “save point”. Nada entra sem PR + CI verde.

E depois?

Bash resolve o MVP de hoje

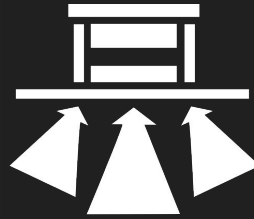
orquestradores escalam quando o projeto cresce



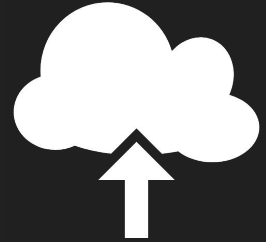
Quando migrar?



Muitas amostras
/ Experimentos



Paralelismo



Trabalhando na
nuvem

Exemplo

```
process RNASEQ_PIPELINE {  
  container 'usuario/rnaseq-pipeline:latest'  
  
  input:  path fastq_file  
  output: path 'results/'  
  
  script:  
  """  
  bash /app/rnaseq_pipeline.sh $fastq_file  
  """  
}
```

- **container:** a imagem que construímos hoje, reutilizada.
- **script:** o mesmo rnaseq_pipeline.sh.
- **Portabilidade:** trocar o experimento não muda a lógica.

Concluindo

1. **Reprodutibilidade se constrói:** código + ambiente empacotados numa imagem Docker versionável.
2. **CI/CD é a rede de segurança:** cada push reconstrói e testa o pipeline de ponta a ponta.
3. **O fluxo por PR protege a main:** nada entra sem check verde.